



Trabajo Práctico N°2

IA 5.2 Computer Vision

1° Cuatrimestre 2026

Sistema de Detección y Clasificación de Razas de Perros

1. Introducción

El objetivo de este trabajo práctico es consolidar y aplicar los conocimientos adquiridos durante las primeras unidades de la materia Computer Vision, integrando conceptos teóricos con la implementación de un sistema completo de detección y clasificación de razas de perros utilizando técnicas modernas de Deep Learning.

Se espera que el estudiante no solo comprenda cómo construir un sistema funcional, sino también por qué se toman determinadas decisiones de diseño, analizando los trade-offs entre precisión, velocidad de inferencia, consumo de memoria y complejidad computacional.

El proyecto se desarrollará de manera incremental, comenzando por un sistema de búsqueda por similitud basado en embeddings y evolucionando hasta un pipeline completo capaz de detectar y clasificar perros en imágenes complejas.

El pipeline implementado deberá evolucionar progresivamente siguiendo el siguiente enfoque:

Embeddings → Búsqueda por similitud → Clasificación → Detección → Pipeline completo

Deberá realizarse un fork del repositorio provisto por la cátedra como template del trabajo práctico.

2. Objetivo

Completar e integrar los componentes fundamentales de un pipeline moderno de Computer Vision para la detección y clasificación de razas de perros utilizando la infraestructura provista por la cátedra capaz de:

- Identificar razas de perros mediante búsqueda por similitud.
- Extraer embeddings utilizando modelos pre-entrenados.
- Construir una base vectorial para recuperación eficiente de imágenes.
- Clasificar razas de perros utilizando modelos entrenados.
- Detectar perros en imágenes complejas.
- Clasificar automáticamente cada perro detectado.
- Generar anotaciones automáticas en formatos estándar utilizados en Computer Vision.

Alcances

Dataset principal:

[70 Dog Breeds Image Dataset \(Kaggle\)](#)

Repositorio base:

El estudiante deberá realizar un fork del repositorio provisto por la cátedra.

3. Alcance del sistema

El trabajo deberá cubrir el pipeline completo de un sistema moderno de identificación y clasificación de razas de perros.

Etapas 1: Buscador de Imágenes por Similitud

El sistema deberá:

- Extraer embeddings de imágenes.
- Construir una base de datos vectorial.
- Recuperar imágenes similares.
- Clasificar razas utilizando búsqueda por similitud.

Etapas 2: Clasificación Supervisada

El sistema deberá:

- Entrenar modelos de clasificación.
- Comparar distintos enfoques.
- Analizar métricas de desempeño.

Etapas 3: Detección y Clasificación

El sistema deberá:

- Detectar perros en imágenes complejas.
- Clasificar cada perro detectado.
- Mostrar resultados visuales sobre la imagen original.

Etapas 4: Evaluación y Optimización

El sistema deberá:

- Se recomienda desacoplar cada componente para facilitar testing, debugging y extensibilidad.

La cátedra proveerá un repositorio base completamente funcional que incluirá:

- El estudiante deberá realizar un fork del repositorio y completar únicamente las funciones indicadas en cada etapa.

No será necesario desarrollar interfaces gráficas, APIs, contenedores Docker ni configuraciones de infraestructura.

Las funciones a implementar serán evaluadas mediante la correcta integración con el sistema provisto por la cátedra.

Etapa 1

Implementar:

- `extract_embedding(image)`
- `search_similar_images(embedding, top_k)`
- `predict_breed_from_neighbors(results)`

Etapas 2

Implementar:

- `train_classifier()`
- `evaluate_classifier()`
- `extract_custom_embedding(image)`

Etapas 3

Implementar:

- `detect_dogs(image)`
- `classify_detected_dog(crop)`

Etapas 4

Implementar:

- `evaluate_pipeline()`
- `optimize_model()`
- `generate_annotations(folder_path, output_format)`

Cualquier modificación fuera de las funciones indicadas deberá estar debidamente justificada en el informe.

La evaluación se realizará ejecutando el proyecto utilizando exclusivamente los comandos provistos en el repositorio.

4. Alcance del trabajo práctico

El trabajo deberá incluir en la notebook principal o informe técnico la documentación de los siguientes elementos.

Dataset

Se deberá:

- Utilizar el dataset 70 Dog Breeds Image Dataset.
 - Analizar la distribución de clases.
 - Documentar cantidad de imágenes por raza.
 - Definir conjuntos de entrenamiento, validación y prueba.
 - Construir un conjunto independiente para evaluación. (Imágenes descargadas desde internet por ejemplo).
-

Preprocesamiento

Definir y justificar técnicas como:

- Resize.
- Normalización.
- Data Augmentation:
 - Horizontal Flip.
 - Rotación.
 - Blur.
 - Variaciones de brillo.

- Variaciones de contraste.
 - Ruido.
 - Filtrado de imágenes de baja calidad.
-

Etapa 1: Buscador de Imágenes por Similitud

Creación de la Base de Datos Vectorial

Utilice un modelo pre-entrenado en ImageNet para generar embeddings.

Ejemplos:

- ResNet50
- EfficientNet
- ConvNeXt

Los embeddings deberán almacenarse en una base vectorial.

Estructura sugerida:

id_imagen: str

embedding: list[float]

path: str

breed: str

metadata: dict

Opciones

- PostgreSQL + pgvector
- ChromaDB
- FAISS
- MongoDB

Se deberá implementar búsqueda por similitud utilizando:

- Cosine Similarity
- Distancia Euclidiana (L2)

Aplicación Gradio

Input:

- Imagen de un perro.

Output:

- Imagen consultada.
- Top 10 imágenes similares.
- Raza predicha.

Evaluación

Se deberá construir un conjunto de prueba independiente.

Para cada imagen:

- Ejecutar búsqueda.
- Recuperar Top-10 resultados.
- Calcular NDCG@10.

Se deberá justificar el resultado obtenido.

Etapla 2: Entrenamiento y Comparación de Modelos

Modelo A (Obligatorio)

Realizar fine-tuning de un modelo:

- ResNet18 pre-entrenado.

Modelo B (Opcional - Recomendado)

Diseñar y entrenar una CNN propia.

Evaluación

Se deberán reportar:

- Accuracy.
- Precision.
- Recall (Sensibilidad).
- Specificity (Especificidad).
- F1-Score.

Además se deberá incluir:

- Matriz de confusión.
- Curvas de entrenamiento.

- Comparación entre modelos.

Integración en la Aplicación

La aplicación deberá permitir seleccionar dinámicamente el modelo utilizado para generar embeddings y realizar la búsqueda.

Ejemplo:

- ResNet18 Fine-Tuned
 - CNN Custom
-

Etapas 3: Pipeline de Detección y Clasificación

Detección de Objetos

Utilizar un modelo YOLO pre-entrenado.

Ejemplo:

- YOLOv8n

No es necesario entrenar el detector.

El sistema deberá funcionar con:

- Un perro.
- Múltiples perros.
- Escenas complejas con otros objetos.

Pipeline Completo

Flujo esperado:

1. Usuario carga una imagen.
2. YOLO detecta todos los perros.
3. Se generan bounding boxes.
4. Se recortan las regiones detectadas.
5. Cada recorte es clasificado.
6. Se generan las predicciones finales.

La aplicación deberá mostrar:

- Bounding boxes.
 - Raza predicha.
 - Score de confianza.
-

Etapas 4: Evaluación, Optimización y Herramientas de Anotación

Evaluación del Pipeline

Se deberá construir un conjunto de prueba compuesto por al menos:

- 10 imágenes complejas.

Cada imagen deberá estar anotada manualmente indicando:

- Bounding boxes.

- Raza correspondiente.

Se deberán calcular:

- mAP
- IoU
- Precision
- Recall
- F1-Score

Además se deberá realizar un análisis detallado de los resultados obtenidos.

Optimización de Modelos

Elegir una de las siguientes estrategias.

Opción 1: Cuantización

Reducir la precisión de los pesos:

- FP32 → INT8

Opción 2: Exportación Optimizada

Convertir los modelos a:

- ONNX
- TensorRT

Comparar:

- Tiempo de inferencia.

- Uso de memoria.
 - Precisión.
-

Script de Anotación Automática

Desarrollar un script que:

- Procese una carpeta de imágenes.
- Detecte perros.
- Clasifique razas.
- Genere anotaciones automáticamente.

Formatos requeridos:

YOLOv5

class x_center y_center width height

COCO

```
{  
  
  "images": [],  
  
  "annotations": [],  
  
  "categories": []  
}
```

Interfaz de Usuario

Se utilizará el frontend provisto por la cátedra basado en Gradio.

Inputs

- Imagen.

Outputs

Etapas

Etapas

- Imagen consultada.
- Top 10 imágenes similares.
- Raza predicha.

Etapas

- Imagen original.
 - Bounding boxes.
 - Etiquetas de raza.
 - Scores de confianza.
-

Configuración

No se permite hardcodear configuraciones.

Debe utilizarse:

- Variables de entorno.
- Archivos .env.

Ejemplos:

- Modelo seleccionado.
- Threshold de similitud.
- Cantidad de vecinos.
- Paths.
- Configuración de YOLO.

5. Entregables

Los alumnos deberán entregar un repositorio propio utilizando como base el template provisto por la cátedra.

Se deberá entregar:

- Repositorio Git privado.
- Pull Request abierto contra el repositorio original provisto por la cátedra.
- Informe en IPYNB.
- Notebook de experimentación.

Para que el trabajo práctico se considere aprobado, el sistema debe andar sin errores corriendo los siguientes comandos :

docker compose build

docker compose up

Se debe entregar un enlace con el pull request que será evaluado por la cátedra sin mergear a main o el branch principal.

6. Buenas prácticas esperadas

Se valorará especialmente:

- Código modular.
 - Separación por capas.
 - Manejo de errores.
 - Logging.
 - Type hints.
 - Tests básicos.
 - Reproducibilidad de experimentos.
-

7. Documentación

Debe incluirse:

- Explicación completa del pipeline.
- Justificación de los modelos elegidos.
- Proceso de entrenamiento.
- Hiperparámetros utilizados.
- Resultados obtenidos.
- Problemas encontrados y soluciones implementadas.
- Comparación entre enfoques.

8. Evaluación

Se evaluará:

- **Funcionalidad.**
- **Calidad del código.**
- **Arquitectura.**
- **Correcta implementación de las etapas.**
- **Manejo de errores.**
- **Calidad de la documentación.**
- **Justificación técnica.**
- **Resultados experimentales.**
- **Participación (commits y PRs).**
- **Defensa oral.**

9. Integrantes

Máximo 2 personas.

10. Fecha de entrega

Sábado 27/06 hasta las 23:59 (vía campus).

- Solo se considerarán commits hasta esa hora
- Se recomienda trabajar de forma incremental (no un único commit final)

11. Recomendaciones finales (clave para aprender bien)

- No copiar implementaciones sin comprenderlas.
- Visualizar embeddings mediante PCA o t-SNE.
- Analizar falsos positivos y falsos negativos.
- Comparar arquitecturas.
- Evaluar imágenes fuera del dataset.
- Medir todas las métricas reportadas.
- Analizar el impacto de las técnicas de optimización.
- Documentar adecuadamente cada decisión tomada durante el desarrollo.
- En el caso de utilizar IA, que sea de forma moderada y razonable.